

Chapter 3

GPIO

Experiment 1 – Lighting up an LED

(Using Addressing)

```
#include <stdint.h>

#define RCC_BASE 0x40021000
#define APB2ENR_OFFSET 0x18
#define GPIOA_BASE 0x40010800
#define GPIOA_CRL_OFFSET 0x00
#define GPIOA_ODR_OFFSET 0x0C

#define RCC_APB2ENR (*(volatile uint32_t *)
(RCC_BASE+APB2ENR_OFFSET))
#define GPIOA_CRL (*(volatile uint32_t *)
(GPIOA_BASE+GPIOA_CRL_OFFSET))
#define GPIOA_ODR (*(volatile uint32_t *)
(GPIOA_BASE+GPIOA_ODR_OFFSET))

void GPIO_Clock_Enable(void);
void GPIO_Pin_Init(void);

void GPIO_Clock_Enable(void)
{
    RCC_APB2ENR |= 1UL<<2; //0x0100
}

void GPIO_Pin_Init(void)
{
    GPIOA_CRL &= ~(15UL<<20);
    GPIOA_CRL |= 3UL<<20;
}

int main(void)
{
    GPIO_Clock_Enable();
    GPIO_Pin_Init();

    GPIOA_ODR |= 1UL<<5;
    while(1);
}
```

(Using Header File)

```
#include "stm32f10x.h" // Device header
```

```
void GPIO_Clock_Enable(void);
```

```
void GPIO_Pin_Init(void);
```

```
int main(void)
```

```
{
```

```
    GPIO_Clock_Enable();
```

```
    GPIO_Pin_Init();
```

```
    GPIOA->ODR |= 1UL<<5;
```

```
    while(1);
```

```
}
```

```
void GPIO_Clock_Enable(void)
```

```
{
```

```
    RCC->APB2ENR |= 1UL<<2; //0x0100
```

```
}
```

```
void GPIO_Pin_Init(void)
```

```
{
```

```
    GPIOA->CRL &= ~(15UL<<20);
```

```
    GPIOA->CRL |= 3UL<<20;
```

```
}
```

(Using Assembly)

```
; Memory addresses of GPIO port A and RCC (reset and clock control) data structure
```

```
GPIOA_BASE      EQU 0x40010800
```

```
RCC_BASE        EQU 0x40021000
```

```
; Byte offset of each register in GPIO and RCC
```

```
RCC_APB2ENR     EQU 0x18
```

```
GPIO_CRL         EQU 0x00
```

```
GPIO_ODR         EQU 0x0C
```

```
AREA main, CODE, READONLY
```

```
EXPORT _main ; make _main visible to Linker
```

```
ENTRY
```

```
_main PROC
```

```
    ; Enable the clock to GPIO port A
```

```
    ; Load address of reset and clock control (RCC)
```

```
    LDR r2, =RCC_BASE          ; Load base address of RCC into r2
```

```
    LDR r1, [r2, #RCC_APB2ENR] ; Load the value of RCC_APB2ENR into r1
```

```
    ORR r1, r1, #(1 << 2)      ; Set bit 2 (IOPAEN) to enable GPIOA clock
```

```

    STR r1, [r2, #RCC_APB2ENR] ; Store the updated value back
into RCC_APB2ENR

    ; Load GPIO port A base address
    LDR r3, =GPIOA_BASE ; Load the base address of
GPIOA into r3

    ; Configure pin PA5 as general-purpose output push-pull
    ; with max speed of 50 MHz
    LDR r1, [r3, #GPIO_CRL] ; Load the value of GPIOA_CRL
into r1
    BIC r1, r1, #(15 << 20) ; Clear bits 23:20 belonging
to pin 5
    ORR r1, r1, #(3 << 20) ; Set bits 23:20 to 0011,
configuring PA5
                                ; as output with max speed 50 MHz
    STR r1, [r3, #GPIO_CRL] ; Store the updated value back
into GPIOA_CRL

    ; Set PA5 high to turn on the LED
    LDR r1, [r3, #GPIO_ODR] ; Load the value of GPIOA_ODR
into r1
    ORR r1, r1, #(1 << 5) ; Set bit 5 to 1, which sets
PA5 high
    STR r1, [r3, #GPIO_ODR] ; Store the updated value back
into GPIOA_ODR

stop
    B stop ; Infinite loop to keep the
program running
    ENDP

END

```

Experiment 2 – Push Button

(Using Addressing)

```

#define RCC_APB2ENR (*(volatile uint32_t *)0x40021018)
#define GPIOB_CRL    (*(volatile uint32_t *)0x40010C00)
#define GPIOC_CRH    (*(volatile uint32_t *)0x40011004)
#define GPIOB_ODR    (*(volatile uint32_t *)0x40010C0C)
#define GPIOC_IDR    (*(volatile uint32_t *)0x40011008)

void delay(uint32_t count) {
    for(uint32_t i = 0; i < count; ++i) {
        // NOP
    }
}

```

```

void setup() {
    // Enable clocks for GPIOB and GPIOC
    RCC_APB2ENR |= (1<<3); // GPIOB
    RCC_APB2ENR |= (1<<4); // GPIOC

    // Set PB0 as output (push-pull, max speed 50 MHz)
    GPIOB_CRL &= ~(15UL<<0);
    GPIOB_CRL |= 3UL<<0;

    // Set PC15 as input
    GPIOC_CRH &= ~(15UL<<28);
    GPIOC_CRH |= 4UL<<28;
}

uint8_t debounce(uint8_t last) {
    uint8_t current = (GPIOC_IDR & (1<<15)) ? 1 : 0;
    if (last != current) {
        delay(50000);
        current = (GPIOC_IDR & (1<<15)) ? 1 : 0;
    }
    return current;
}

int main() {
    uint8_t lastButton = 0;
    uint8_t currentButton = 0;
    uint8_t ledOn = 0;

    setup();

    while (1) {
        currentButton = debounce(lastButton);
        if (lastButton == 0 && currentButton == 1) {
            ledOn = !ledOn;
        }
        lastButton = currentButton;
        if (ledOn) {
            GPIOB_ODR |= (1<<0);
        } else {
            GPIOB_ODR &= ~(1<<0);
        }
    }
}

```

(Using Header File)

```
#include "stm32f10x.h" // Device header
```

```

void delay(uint32_t count) {
    for(uint32_t i = 0; i < count; ++i) {
        // NOP
    }
}

```

```

    }
}

void setup() { // Enable clocks for GPIOB and GPIOC
    RCC->APB2ENR |= (1<<3); // GPIOB
    RCC->APB2ENR |= (1<<4); // GPIOC

    // Set PB0 as output (push-pull, max speed 50 MHz)
    GPIOB->CRL &= ~(15UL<<0);
    GPIOB->CRL |= 3UL<<0;

    // Set PC15 as input
    GPIOC->CRH &= ~(15UL<<28);
    GPIOC->CRH |= 4UL<<28;
}

uint8_t debounce(uint8_t last) {
    uint8_t current = (GPIOC->IDR & (1<<15)) ? 1 : 0;
    if (last != current) {
        delay(50000);
        current = (GPIOC->IDR & (1<<15)) ? 1 : 0;
    }
    return current;
}

int main() {
    uint8_t lastButton = 0;
    uint8_t currentButton = 0;
    uint8_t ledOn = 0;

    setup();

    while (1) {
        currentButton = debounce(lastButton);
        if (lastButton == 0 && currentButton == 1) {
            ledOn = !ledOn;
        }
        lastButton = currentButton;
        if (ledOn) {
            GPIOB->ODR |= (1<<0);
        } else {
            GPIOB->ODR &= ~(1<<0);
        }
    }
}

```

Experiment 3 - 4 x 3 Keypad

```
#define RCC_APB2ENR (*(volatile uint32_t *) 0x40021018)
#define GPIOA_CRL    (*(volatile uint32_t *) 0x40010800)
#define GPIOA_IDR    (*(volatile uint32_t *) 0x40010808)
#define GPIOA_ODR    (*(volatile uint32_t *) 0x4001080C)

int row, col;

unsigned char key_map [4][3] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'},
    {'*', '0', '#'}
};

void enable_clock() {
    // Enable port A clock
    RCC_APB2ENR |= 1UL<<2;
}

char input_string[100] = {0}; // Initialize input string
int input_index = 0;          // Initialize input index

void add_to_string(char c) {
    // Ensure there is space for the new character and the
    null terminator
    if (input_index < sizeof(input_string) - 1) {
        input_string[input_index] = c; // Add character at the
current index
        input_index++;                // Move to the next
position
        input_string[input_index] = '\0'; // Null terminate
the string
    }
}

void set_output_pins() {
    // Set PA0, PA1, PA2, PA3 as output (R1, R2, R3, R4)
    GPIOA_CRL &= ~(15UL<<0); // Erase PA0 CNF and MODE
    GPIOA_CRL |= (3UL<<0);    // Set PA0 as Output

    GPIOA_CRL &= ~(15UL<<4); // Erase PA1 CNF and MODE
    GPIOA_CRL |= (3UL<<4);    // Set PA1 as Output

    GPIOA_CRL &= ~(15UL<<8); // Erase PA2 CNF and MODE
    GPIOA_CRL |= (3UL<<8);    // Set PA2 as Output

    GPIOA_CRL &= ~(15UL<<12); // Erase PA3 CNF and MODE
    GPIOA_CRL |= (3UL<<12);   // Set PA3 as Output
}
```

```

}

void set_input_pins() {
    // Set PA4, PA5, PA6 as Input (C1, C2, C3)
    GPIOA_CRL &= ~(15UL<<16); // Erase PA4 CNF and MODE
    GPIOA_CRL |= (4UL<<16);    // Set PA4 as Input

    GPIOA_CRL &= ~(15UL<<20); // Erase PA5 CNF and MODE
    GPIOA_CRL |= (4UL<<20);    // Set PA5 as Input

    GPIOA_CRL &= ~(15UL<<24); // Erase PA6 CNF and MODE
    GPIOA_CRL |= (4UL<<24);    // Set PA6 as Input
}

void set_input_pins() {
    // Set PA4, PA5, PA6 as Input (C1, C2, C3)
    GPIOA_CRL &= ~(15UL<<16); // Erase PA4 CNF and MODE
    GPIOA_CRL |= (4UL<<16);    // Set PA4 as Input

    GPIOA_CRL &= ~(15UL<<20); // Erase PA5 CNF and MODE
    GPIOA_CRL |= (4UL<<20);    // Set PA5 as Input

    GPIOA_CRL &= ~(15UL<<24); // Erase PA6 CNF and MODE
    GPIOA_CRL |= (4UL<<24);    // Set PA6 as Input
}

void get_column() {
    // Check whether PA4 is 0 or not
    if((GPIOA_IDR & 1UL<<4) == 0) {
        col = 0;
    }
    // Check whether PA5 is 0 or not
    else if((GPIOA_IDR & 1UL<<5) == 0) {
        col = 1;
    }
    // Check whether PA6 is 0 or not
    else if((GPIOA_IDR & 1UL<<6) == 0) {
        col = 2;
    }
    // PA4, PA5, PA6 are all 1. No button pressed.
    else { col = -1; }
}

void get_row() {
    // Set PA0 as 0 and check if all columns are 1
    GPIOA_ODR &= ~(15UL<<0);
    GPIOA_ODR |= (14UL<<0);
    if(GPIOA_IDR & (7UL<<4) != 112) {row = 0;}
    // Set PA0 as 0 and check if all columns are 1
    GPIOA_ODR &= ~(15UL<<0);
    GPIOA_ODR |= (13UL<<0);
}

```

```

        if(GPIOA_IDR & (7UL<<4) != 112) {row = 1;}
        // Set PA0 as 0 and check if all columns are 1
        GPIOA_ODR &= ~(15UL<<0);
        GPIOA_ODR |= (11UL<<0);
        if(GPIOA_IDR & (7UL<<4) != 112) {row = 2;}
        // Set PA0 as 0 and check if all columns are 1
        GPIOA_ODR &= ~(15UL<<0);
        GPIOA_ODR |= (7UL<<0);
        if(GPIOA_IDR & (7UL<<4) != 112) {row = 3;}
    }

int main() {
    enable_clock();
    set_output_pins();
    set_input_pins();

    while(1) {
        get_column();
        if (col == -1) {
            continue;
        }
        get_row();
        add_to_string(key_map[row][col]);
    }
}

```